

EdgeHealth-ML

端侧多模态人体活动识别系统

完整项目分析与代码详解

吴金骏 | WU, Jinjun

Hong Kong University of Science and Technology

2026 年 6 月 19 日

摘要：本文档对 EdgeHealth-ML 项目进行完整技术分析，涵盖项目背景、模型架构（Vision+IMU 双分支融合网络）、数据处理流水线（STFT 时频谱图生成）、训练流程（30 epochs，最佳准确率 87.61%）、INT8 量化与 ONNX 部署（ONNX Runtime 推理延迟 4.83ms），以及全部 6 个 Python 源文件的逐行代码详解。项目面向移动/边缘设备的实时人体活动识别，对标 HKUST MINSys Lab 研究方向。

项目概述

背景与目标

EdgeHealth-ML 是一个面向**移动/边缘设备**的**多模态人体活动识别**（Human Activity Recognition, HAR）系统。利用智能手机自带的**加速度计 + 陀螺仪**（IMU 传感器），在设备端实时识别用户的日常活动（走路、上楼、下楼、坐着、站着、躺着），无需联网。项目对标 HKUST MINSys Lab（欧阳晓敏教授）的研究方向。

技术亮点

1. **双分支异构融合**： Vision（2D CNN 处理频谱图）+ IMU（1D CNN 处理时序），互补特征通过拼接融合
2. **端到端部署**： PyTorch 训练 → INT8 量化 → ONNX 导出 → ONNX Runtime 推理
3. **轻量化设计**： MobileNetV3-Small 作为视觉主干，总参数量仅 120 万
4. **频谱图转换**： 将 1D 时序信号通过 STFT 转为 2D 时频谱图，复用成熟图像分类架构
5. **量化无损**： INT8 动态量化后准确率仅下降 0.03%，ONNX Runtime 推理加速 11 倍

数据集

UCI HAR Dataset:

- 30 名受试者，佩戴智能手机执行 6 种活动
- 传感器采样率 50Hz，每个样本 128 个时间步（2.56 秒窗口）
- 训练集 7,352 样本，测试集 2,947 样本
- 6 类活动： walking, walking_upstairs, walking_downstairs, sitting, standing, laying

模型架构

整体结构

模型由 Vision 分支 (MobileNetV3-Small) 处理时频谱图、IMU 分支 (3 层 1D-CNN) 处理原始传感器信号、融合 MLP 分类三部分组成。

Vision 分支 (VisionEncoder)	IMU 分支 (IMUEncoder)
MobileNetV3-Small	3 层 1D-CNN
输入: (B, 3, 224, 224)	输入: (B, 6, 128)
输出: (B, 576)	输出: (B, 128)

Fusion MLP
concat(576+128=704)
→ Linear(704→256)
→ BN + ReLU + Dropout
→ Linear(256→6)

6 类活动分类输出

VisionEncoder: 频谱图编码器

- 主干网络: MobileNetV3-Small, 使用 ImageNet 预训练权重 (迁移学习)
- 输入: 3 通道时频谱图 (B, 3, 224, 224)
- 处理流程: backbone.features (去掉分类头) → AdaptiveAvgPool2d(1) (全局平均池化)
- 输出: 576 维特征向量 (B, 576)

MobileNetV3-Small 使用深度可分离卷积（depthwise separable convolution）和 SE（Squeeze-and-Excitation）注意力模块，在保持较高精度的同时大幅减少计算量。

IMUEncoder：时序信号编码器

- **结构：** 3 层 1D 卷积 + BatchNorm + ReLU
- **输入：** 原始 6 通道传感器信号 $(B, 6, 128)$ ——3 轴加速度 + 3 轴陀螺仪，128 个时间步
- **维度变化：**

```
(B, 6, 128)  → Conv1d(6→64, k=5, s=2)  → (B, 64, 64)
(B, 64, 64)  → Conv1d(64→128, k=5, s=2) → (B, 128, 32)
(B, 128, 32) → Conv1d(128→128, k=5, s=2) → (B, 128, 16)
(B, 128, 16) → AdaptiveAvgPool1d(1)      → (B, 128)
```

- **卷积核大小 5：** 捕捉 5 个时间步（0.1 秒 @ 50Hz）的局部运动模式
- **步长 2：** 逐步降采样，增大感受野

融合策略：晚期融合

- **融合方式：** Concat 拼接 → 704 维 $(576 + 128)$
- **MLP 结构：** Linear(704→256) → BatchNorm → ReLU → Dropout(0.3) → Linear(256→6)
- **Dropout(0.3)：** 唯一的正则化手段，防止过拟合

为什么选择晚期融合？ 两个模态的物理意义完全不同——频谱图是频域二维图像，IMU 是时域一维信号。先让各自分支独立提取模态特定特征，再在高维语义层融合，避免模态间干扰。

数据处理流水线

原始数据格式

UCI HAR Dataset 的原始格式是空格分隔的纯文本文件：

- `body_acc_x_train.txt`: 身体加速度 X 轴, 形状 (7352,128)
- `y_train.txt`: 训练标签, 1-6 映射到 0-5

共 9 个信号文件 (6 个身体传感器 + 3 个总加速度), 加 2 个标签文件。

`_load_txt()` — 文本读取

```
1 def _load_txt(path):
2     with open(path, "r") as f:
3         return np.array(
4             [[float(x) for x in line.strip().split()] for line in
5              f])
```

逐行读取 → 去除首尾空白 → 按空格分割 → 转为浮点数 → 返回 $(N, 128)$ 的 numpy 数组。

双模态数据生成

每个原始样本的传感器数据被处理成两种互补的输入形式:

模态 1: 时频谱图 (Vision 分支输入)

加速度信号 $(3, 128)$ → STFT → | 幅度 | → $\log(1+x)$ → `resize(224,224)` → $(3, 224, 224)$

STFT 参数: `nperseg=64` (1.28 秒窗口)、`noverlap=48` (75% 重叠)、`window="hann"` (汉宁窗)。

关键设计:

- 3 轴加速度各自独立做 STFT → 3 张谱图沿 channel 维度堆叠
- `log1p` 压缩动态范围: 原始幅度 $\approx [0, 100]$, 取 `log` 后 $\approx [0, 5]$, 利于神经网络学习
- `scipy.ndimage.zoom` (`order=1`, 双线性插值) 缩放到 224×224
- 最后 `Normalize(mean=0.5, std=0.5)` 映射到 $[-1, 1]$

模态 2: 原始 IMU 信号 (IMU 分支输入)

- 3 轴加速度 + 3 轴陀螺仪 = 6 通道 $\times$ 128 时间步 $\rightarrow$ 直接作为 tensor
- 不做额外处理, 保留原始时序信息
- 加速度反映姿态和运动方向, 陀螺仪反映旋转角速度

HARDataLoader 类

继承 `torch.utils.data.Dataset`, `__getitem__` 返回三元组 (spectrogram, imu_signal, label)。

```

1 class HARDataLoader(Dataset):
2     def __getitem__(self, idx):
3         # 模态 1: 时频谱图
4         acc_signals = np.stack([body_acc_x[idx], ...], axis=0) #
5             (3, 128)
6         spec = _build_spectrogram_image(acc_signals, 224) #
7             (3, 224, 224)
8         spec = torch.from_numpy(spec)
9         spec = spec_norm(spec) # Normalize
10
11         # 模态 2: 原始 IMU 信号
12         imu = np.stack([body_acc_x[idx], ..., body_gyro_z[idx]],
13             axis=0)
14         imu = torch.from_numpy(imu).float() # (6, 128)
15
16         return spec, imu, int(labels[idx])

```

create_dataloaders() 工厂函数

```

1 def create_dataloaders(data_dir, batch_size=32):
2     train_loader = DataLoader(train_ds, batch_size=batch_size,
3         shuffle=True)
4     test_loader = DataLoader(test_ds, batch_size=batch_size,
5         shuffle=False)
6     return train_loader, test_loader, len(LABELS)

```

`shuffle=True` 训练时打乱, `False` 测试时保持顺序; `num_workers=0` 单进程 (Windows 兼容)。

训练流程

超参数配置

参数	值	说明
优化器	AdamW	weight_decay=10 ⁻⁴
初始学习率	0.001	AdamW 初始步长
学习率调度	CosineAnnealingLR	余弦衰减到 0
损失函数	CrossEntropyLoss	多分类交叉熵
Batch Size	32	每批 32 个样本
Epochs	30	完整遍历 30 次
设备	CPU	Intel Core（无 GPU）

余弦退火学习率

学习率按余弦曲线从初始值平滑衰减到 0：

$$\text{lr}(t) = \frac{\text{lr}_0}{2} \left(1 + \cos \frac{t\pi}{T} \right)$$

t 为当前 epoch， $T = 30$ 为总 epoch 数。后期极小学习率帮助模型收敛到更优局部最小值。

训练循环

每个 epoch 执行：

- `model.train()`：启用 Dropout 和 BatchNorm 训练模式
- `train_one_epoch()`：遍历训练集，前向 → 计算损失 → 反向传播 → 更新权重
- `evaluate()`：测试集评估（`@torch.no_grad()` 禁用梯度计算）
- `scheduler.step()`：衰减学习率
- 若 `test_acc > best_acc`，保存 `best_model.pth`

训练日志（完整 30 epochs）

Epoch	Train Loss	Train Acc	Test Acc
1	0.5088	0.775	0.251 *
2	0.3534	0.847	0.707 *
3	0.3119	0.865	0.713 *
4	0.2753	0.889	0.687
5	0.2496	0.894	0.649
6	0.2205	0.910	0.717 *
7	0.1979	0.918	0.692
8	0.1950	0.918	0.716
9	0.1621	0.936	0.795 *
10	0.1293	0.948	0.820 *
11	0.1083	0.959	0.730
12	0.0947	0.963	0.832 *
13	0.0731	0.974	0.799
14	0.0524	0.980	0.785
15	0.0394	0.986	0.841 *
16	0.0318	0.988	0.795
17	0.0270	0.991	0.806
18	0.0138	0.995	0.861 *
19	0.0116	0.997	0.842
20	0.0133	0.995	0.835
21	0.0080	0.998	0.800
22	0.0030	1.000	0.854
23	0.0040	0.999	0.827
24	0.0054	0.999	0.847
25	0.0024	0.999	0.876 *
26	0.0021	1.000	0.871
27	0.0013	1.000	0.871
28	0.0017	1.000	0.874
29	0.0012	1.000	0.873
30	0.0017	1.000	0.873

表 1: 训练日志（* 表示打破历史最佳测试准确率）

最终评估结果

最佳模型在 Epoch 25，测试准确率 **87.61%**。

活动类别	Precision	Recall	F1	样本数
walking	0.9841	1.0000	0.9920	496
walking_upstairs	0.9638	0.9618	0.9628	471
walking_downstairs	0.9758	0.9595	0.9676	420
sitting	0.7140	0.6456	0.6781	491
standing	0.7522	0.8158	0.7827	532
laying	0.8887	0.8920	0.8903	537
总体	0.8761			2947

表 2: 分类报告（测试集）

结论:

- 动态活动（walking 系列）识别近乎完美——walking 100% 召回率，上下楼 >96%
- 静态活动（sitting/standing）存在混淆——两者传感器模式极为相似，这是 HAR 经典难题
- laying 识别较好（89%）——躺下时加速度方向与直立明显不同

混淆矩阵

	walking	up	down	sit	stand	lay
walking	496	0	0	0	0	0
walking_up	8	453	10	0	0	0
walking_down	0	17	403	0	0	0
sitting	0	0	0	317	134	40
standing	0	0	0	78	434	20
laying	0	0	0	49	9	479

主要混淆: sitting ↔ standing（212 样本）、sitting → laying（40 样本）。

模型优化：量化与 ONNX 部署

FP32 ONNX 导出

训练好的 PyTorch 模型（.pth）只能被 PyTorch 加载。ONNX 导出使其可在任何支持 ONNX Runtime 的设备运行（手机、嵌入式等）。

导出原理：

1. 用随机数据（dummy input）运行模型，`torch.export` 追踪所有算子调用
2. PyTorch 计算图转为 ONNX 中间表示（IR）
3. 序列化为 .onnx 文件

关键参数：

- `dynamic_axes={0: "batch"}`: batch 维度动态化，推理时可传入任意 batch size
- `opset_version=18`: 匹配 PyTorch 2.12 的 ONNX 算子集

INT8 动态量化

原理：

原始： 权重(FP32) × 输入(FP32) = 输出(FP32)

量化： [权重(INT8) → 反量化(FP32)] × 输入(FP32) = 输出(FP32)

- 仅量化 `nn.Linear` 和 `nn.Conv1d` 的权重——激活值保持 FP32
- 计算时临时反量化，精度损失极小
- 权重量化减少内存占用约 25%

量化效果对比

ONNX Runtime 推理性能

ONNX Runtime 比 PyTorch 原生推理快 11 倍（4.83ms vs 53.91ms），原因：

1. ONNX Runtime 对计算图做了算子融合和常量折叠优化
2. PyTorch eager 模式有 Python 解释器开销，ONNX Runtime 纯 C++ 执行
3. 批量推理时 ONNX Runtime 多线程并行效率更高

指标	FP32	INT8	变化
准确率	87.61%	87.58%	-0.03%
推理延迟	53.91 ms	52.66 ms	1.02×
模型大小	4,972 KB	4,441 KB	1.12×
参数量	1,235,302	1,053,280	-14.7%

表 3: FP32 vs INT8 量化对比

指标	结果
平均延迟	4.83 ms
标准差	2.53 ms
最小延迟	2.73 ms
最大延迟	21.80 ms
P95 延迟	9.44 ms
吞吐量	207.0 次/秒

表 4: ONNX Runtime 推理基准（FP32 ONNX，100 次迭代）

项目文件结构与代码详解

目录结构

EdgeHealth-ML/	
data/	
download_har.py	# 多镜像下载 UCI HAR 数据集
UCI_HAR_Dataset/	# 原始传感器数据（txt 格式）
models/	
multimodal_model.py	# VisionEncoder + IMUEncoder + Fusion
utils/	
data_loader.py	# STFT 谱图生成 + 数据加载
train.py	# 训练主脚本
quantize_export.py	# 量化 + ONNX 导出
inference_benchmark.py	# ONNX Runtime 推理基准测试
checkpoints/	# 模型权重 + ONNX 文件
runs/	# TensorBoard 训练日志
requirements.txt	# 依赖清单

执行顺序

download_har.py → data_loader.py → multimodal_model.py
↓
inference_benchmark.py ← quantize_export.py ← train.py

data/download_har.py ——数据集下载

功能：下载并解压 UCI HAR Dataset。被 train.py 和 quantize_export.py 调用。

关键设计：

- **多镜像容错**（HAR_URLS 列表）：优先 UCI 官方源，失败则切换 GitHub 镜像
- **幂等性**：如数据集已解压，直接返回路径（if os.path.exists(EXTRACT_DIR)）
- **完整性验证**：下载后用 zipfile.ZipFile 打开验证合法性，损坏则删除重试
- **清理**：解压后删除 zip 文件释放空间

核心函数 download_and_extract()：

```
1 def download_and_extract():
2     if os.path.exists(EXTRACT_DIR):           # 幂等性检查
3         return EXTRACT_DIR
4     for url in HAR_URLS:                       # 遍历镜像
5         try:
6             urlretrieve(url, zip_path)        # 下载
7             with zipfile.ZipFile(zip_path) as test: pass # 验证
8             break
9         except Exception:
10            if os.path.exists(zip_path): os.remove(zip_path)
11            continue                             # 下一个镜像
12    else:
13        sys.exit(1)                             # 全部失败 → 退出
14    with zipfile.ZipFile(zip_path) as zf:
15        zf.extractall(DATA_DIR)                 # 解压
16    os.remove(zip_path)                         # 清理
17    return EXTRACT_DIR
```

utils/data_loader.py ——数据加载与预处理

功能：从原始 txt 文件读取传感器数据，转化为双模态输入。

关键函数：

- `_load_txt(path)`: 读取空格分隔文本 $\rightarrow$ numpy 数组
- `_signal_to_spectrogram(signal_1d, nperseg=64, noverlap=48)`: STFT $\rightarrow$ | 幅度 | $\rightarrow$ log1p $\rightarrow$ 2D 谱图
- `_build_spectrogram_image(signals_3ch, 224)`: 3 通道各做 STFT $\rightarrow$ stack $\rightarrow$ zoom 缩放 $\rightarrow$ normalize
- `_resize_2d(arr, h, w)`: `scipy.ndimage.zoom` 双线性插值

核心类 `HARDataLoader(Dataset)`: `__getitem__` 返回 (spectrogram, imu_signal, label)。

models/multimodal_model.py ——模型架构

功能：定义双分支多模态融合网络的全部组件。

三个核心类：

1. IMUEncoder (1D-CNN 时序编码器) ——行 27–55

- 3 层 `Conv1d`, 每层 `kernel=5, stride=2, padding=2`
- 维度流: $(B, 6, 128) \rightarrow (B, 64, 64) \rightarrow (B, 128, 32) \rightarrow (B, 128, 16) \rightarrow (B, 128)$
- `AdaptiveAvgPool1d(1)` 全局平均池化压缩时间维度

2. VisionEncoder (频谱图编码器) ——行 58–76

- `mobilenet.mobilenet_v3_small(weights="DEFAULT")` 加载预训练
- `backbone.features` 去掉分类头, `AdaptiveAvgPool2d(1)` 全局池化
- 输出 576 维——MobileNetV3-Small 的标准特征维度

3. MultimodalHARModel (融合网络) ——行 79–110

- 包含 `vision_encoder + imu_encoder + fusion MLP`
- Fusion: `Linear(704 $\rightarrow$ 256)  $\rightarrow$  BN  $\rightarrow$  ReLU  $\rightarrow$  Dropout(0.3)  $\rightarrow$  Linear(256 $\rightarrow$ 6)`

- `forward()`: 分别编码 → `torch.cat` 拼接 → 分类

4. `count_parameters()` ——行 116–119

- 统计总参数和可训练参数（本项目 1,235,302 个全部可训练）

`train.py` ——训练脚本

功能：编排完整训练流程。

命令行参数(`parse_args()`): `--epochs 30`、`--batch_size 32`、`--lr 0.001`、`--weight_decay 1e-4`、`--device auto`

关键函数：

- `train_one_epoch()`: 前向 → 损失 → 反向 → 更新，返回平均 loss 和 accuracy
- `evaluate()`: `@torch.no_grad()` 禁用梯度，返回 loss + acc + 所有预测和标签
- `main()`: 4 步流程——
 1. 准备数据（下载 + 创建 `DataLoader`）
 2. 构建模型（实例化 + 打印参数量）
 3. 训练循环（30 epochs + `CosineAnnealingLR` + `TensorBoard` + 保存最佳模型）
 4. 最终评估（`classification_report` + `confusion_matrix`）

优化器选择：AdamW ——与标准 Adam 的区别在于 `weight_decay` 直接作用于权重而非通过梯度，提供更好的正则化效果。

`quantize_export.py` ——量化与 ONNX 导出

功能：将训练好的 PyTorch 模型压缩并导出为跨平台 ONNX 格式。

关键函数：

- `export_onnx(model, filepath)`: `torch.onnx.export()` 导出，支持动态 batch
- `quantize_model(model)`: `torch.quantization.quantize_dynamic()` INT8 动态量化
- `compare_models(fp32, int8, loader)`: 准确率 + 延迟 + 大小三方对比

- `main()`: 4 步——加载数据 → 加载模型 → 导出 FP32 ONNX → INT8 量化 + 对比

兼容性说明: PyTorch 2.12+ 基于 `torch.export` 的 ONNX 导出器不支持量化模型。FP32 ONNX 导出成功 (397 KB), INT8 ONNX 通过 `try/except` 跳过。

`inference_benchmark.py` ——推理性能测试

功能: 用 ONNX Runtime 测试模型在实际部署时的推理速度。

`main()` 流程:

1. 验证 ONNX: `onnx.load()` + `onnx.checker.check_model()`
2. 创建会话: `ort.InferenceSession()` (`CPUExecutionProvider`)
3. 预热: 10 次推理 (排除初始化开销)
4. 基准: 100 次推理, `time.perf_counter()` 高精度计时
5. 统计: 均值、标准差、P95、吞吐量
6. 对比: 若 INT8 ONNX 存在则自动 FP32 vs INT8 对比

从零到一复现

1. 安装依赖:

```
1 pip install torch torchvision numpy scipy scikit-learn onnx
   onnxruntime
```

2. 训练模型 (自动下载数据集):

```
1 python train.py --epochs 30
```

3. 量化并导出 ONNX:

```
1 python quantize_export.py
```

4. 推理性能基准:

```
1 python inference_benchmark.py --onnx checkpoints/model_fp32.
   onnx --iters 100
```

数据流总览

UCI HAR Dataset (.txt 文件)

data_loader.py STFT 谱图 (3,224,224) → VisionEncoder → 576维

原始IMU信号 (6,128) → IMUEncoder → 128维

torch.cat → 704维

Fusion MLP (704→256→6)

6类活动输出

train.py → best_model.pth (87.61%)

quantize_export.py → model_fp32.onnx
→ model_int8.pth

inference_benchmark.py → 4.83ms/次

附录 A：依赖清单

```
1 torch>=2.0.0
2 torchvision>=0.15.0
3 numpy>=1.24.0
4 scipy>=1.10.0
5 scikit-learn>=1.3.0
6 pandas>=2.0.0
7 matplotlib>=3.7.0
8 onnx>=1.14.0
9 onnxruntime>=1.15.0
```

附录 B：6 类活动标签

索引	活动
0	walking
1	walking_upstairs
2	walking_downstairs
3	sitting
4	standing
5	laying

附录 C：6 通道传感器信号

索引	信号名	传感器
0	body_acc_x	加速度计 X 轴
1	body_acc_y	加速度计 Y 轴
2	body_acc_z	加速度计 Z 轴
3	body_gyro_x	陀螺仪 X 轴
4	body_gyro_y	陀螺仪 Y 轴
5	body_gyro_z	陀螺仪 Z 轴